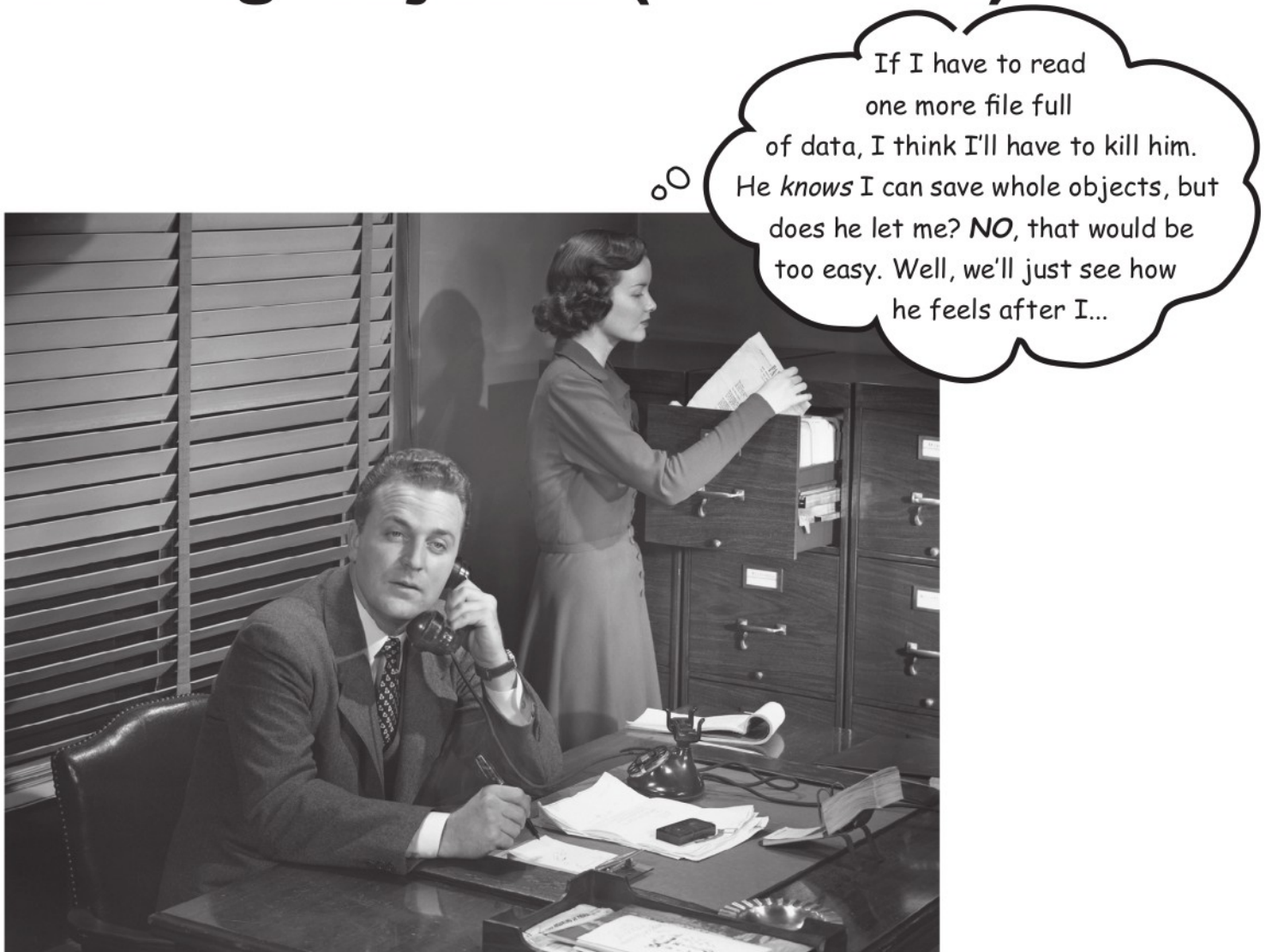
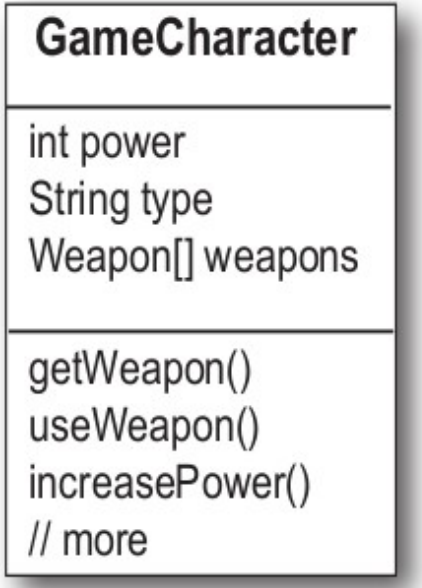
Saving Objects (and Text)



Imagine you have three game characters to save...



If your data will be used by only the Java program that generated it:

① Use serialization

Write a file that holds flattened (serialized) objects. Then have your program read the serialized objects from the file and inflate them back into living, breathing, heap-inhabiting objects.

If your data will be used by *other* programs:

② Write a plain-text file

Write a file, with delimiters that other programs can parse. For example, a tab-delimited file that a spreadsheet or database application can use.

① Option one

Write the three serialized character objects to a file

Create a file and write three serialized character objects. The file won't make sense if you try to read it as text:

```
¨ÌsrGameCharacter
¨%gê8MÛIpowerLjava/lang/
String;[weaponst[Ljava/lang/
String;xp2tlfur[Ljava.lang.String;≠“VÁ
È{Gxptbowtswordtdustsq~»tTrolluq~tb
are handstbig axsq~xtMagicianuq~tspe
llstinvisibility
```

② Option two

Write a plain-text file

Create a file and write three lines of text, one per character, separating the pieces of state with commas:

```
50,Elf,bow, sword,dust
200,Troll,bare hands,big ax
120,Magician,spells,invisibility
```

As a leaf is carried by a stream, whether the stream ends in a lake or in the sea, so too is the output of your program carried by a stream, not knowing if the stream goes to the screen or to a file.

Writing a serialized object to a file

If the file "MyGame.ser" doesn't exist, it will be created automatically.

1 Make a FileOutputStream

```
FileOutputStream fileStream = new FileOutputStream("MyGame.ser");
```

Make a FileOutputStream object. FileOutputStream knows how to connect to (and create) a file.

2 Make an ObjectOutputStream

```
ObjectOutputStream os = new ObjectOutputStream(fileStream);
```

ObjectOutputStream lets you write objects, but it can't directly connect to a file. It needs to be fed a "helper." This is actually called "chaining" one stream to another.

3 Write the object

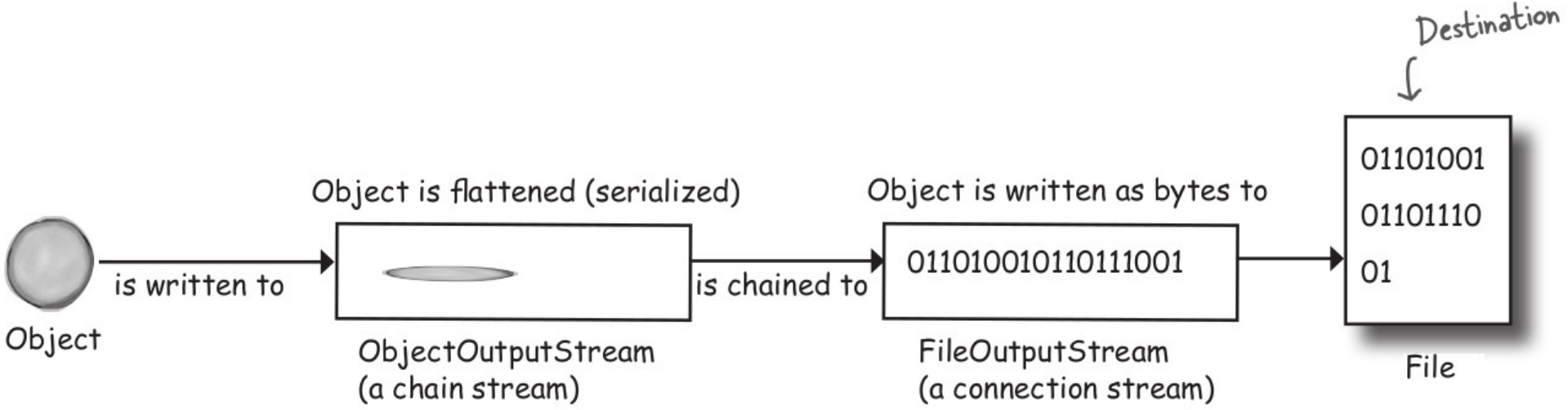
```
os.writeObject(characterOne);
os.writeObject(characterTwo);
os.writeObject(characterThree);
```

Serializes the objects referenced by characterOne, characterTwo, and characterThree, and writes them in this order to the file "MyGame.ser."

4 Close the ObjectOutputStream

```
os.close();
```

Closing the stream at the top closes the ones underneath, so the FileOutputStream (and the file) will close automatically.

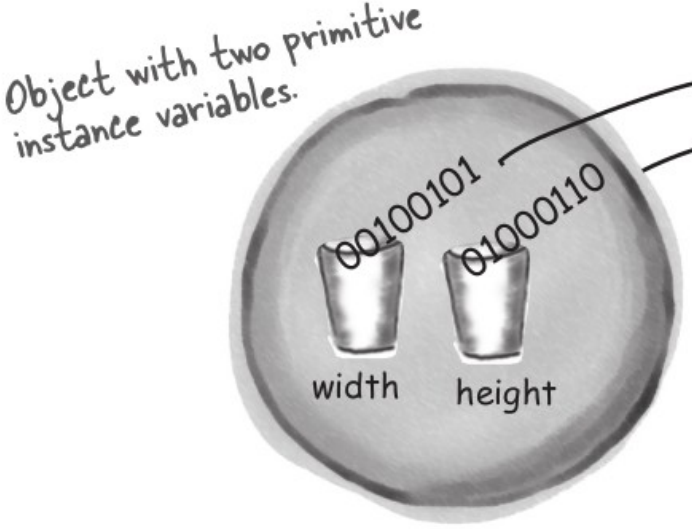

Destination
01101001
01101110
01
Object is flattened (serialized)
Object is written as bytes to
is written to
01101001011011100 1
is chained to
Object
ObjectOutputStream
(a chain stream)
FileOutputStream
(a connection stream)
File

1 Object on the heap

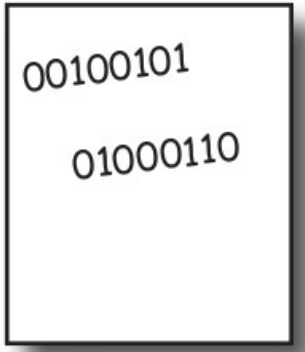
Objects on the heap have state—the value of the object's instance variables. These values make one instance of a class different from another instance of the same class.

2 Object serialized

Serialized objects **save the values of the instance variables** so that an identical instance (object) can be brought back to life on the heap.



The values are sucked out and pumped into the stream.



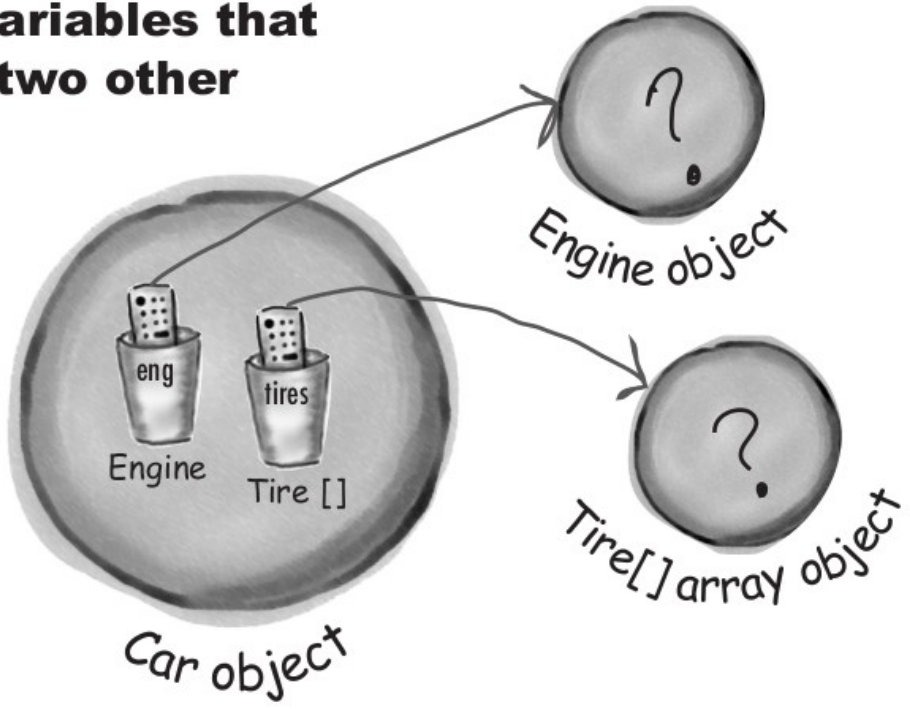
The instance variable values for width and height are saved to the file "foo.ser," along with a little more info the JVM needs to restore the object (like what its class type is).

```
Foo myFoo = new Foo();
myFoo.setWidth(37);
myFoo.setHeight(70);
```

```
FileOutputStream fs = new FileOutputStream("foo.ser");
ObjectOutputStream os = new ObjectOutputStream(fs);
os.writeObject(myFoo);
```

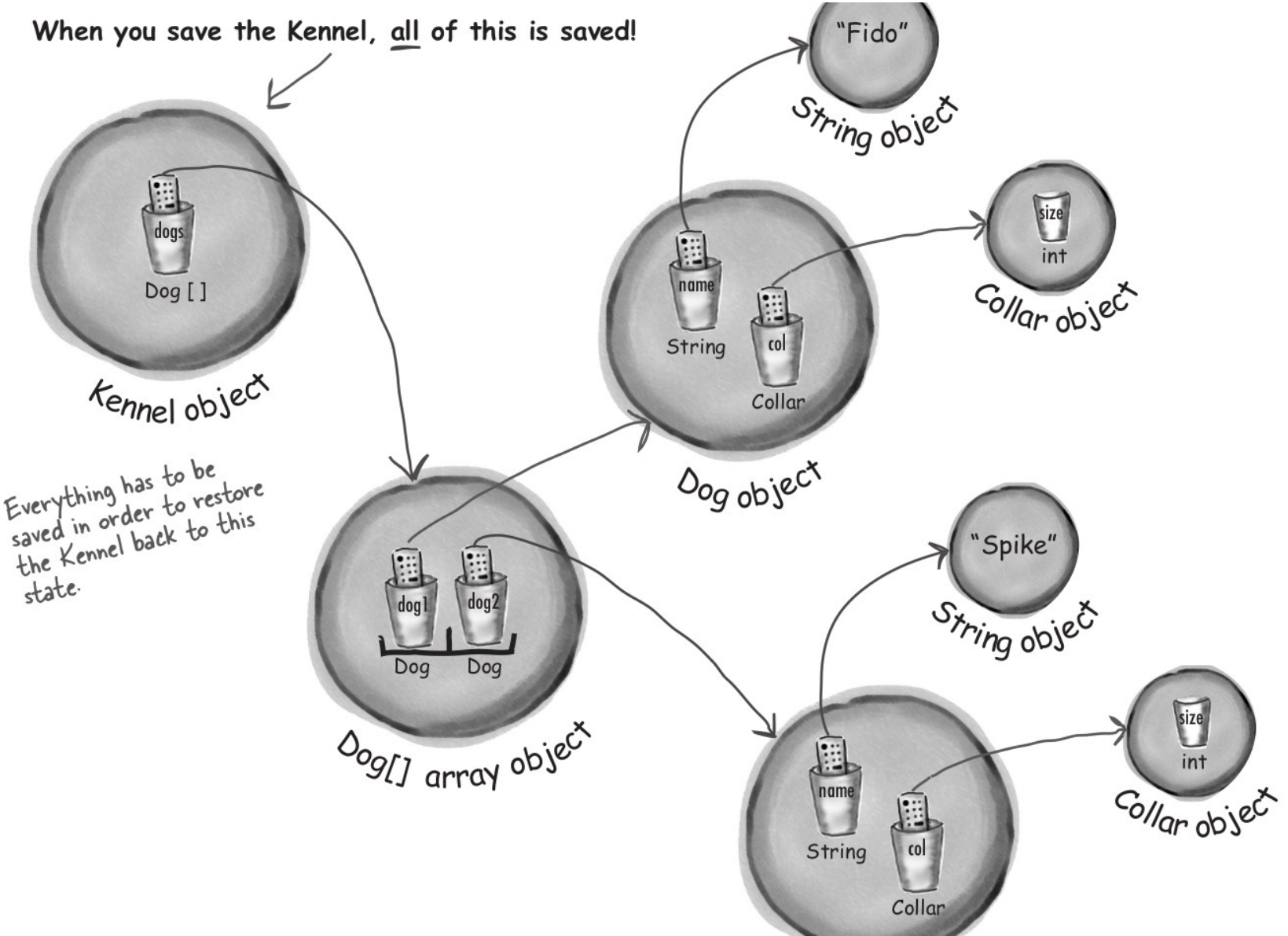
Make a FileOutputStream that connects to the file "foo.ser"; then chain an ObjectOutputStream to it and tell the ObjectOutputStream to write the object.

The Car object has two instance variables that reference two other objects.



Serialization saves the entire **object graph**— all objects referenced by instance variables, starting with the object being serialized.

What does it take to save a Car object?


When you save the Kennel, all of this is saved!
dogs
Dog [ ]
Kennel object
Everything has to be saved in order to restore the Kennel back to this state.
dog1
dog2
Dog
Dog
Dog[] array object
"Fido"
String object
name
String
col
Collar
Dog object
size
int
Collar object
"Spike"
String object
name
String
col
Collar
size
int
Collar object

If you want your class to be serializable, implement Serializable

```
objectOutputStream.writeObject(mySquare);
```

Whatever goes here MUST implement Serializable or it will fail at runtime.

```
import java.io.*;
```

Serializable is in the java.io package, so you need the import.

```
public class Square implements Serializable {
```

No methods to implement, but when you say "implements Serializable," it says to the JVM, "it's OK to serialize objects of this type."

```
  private int width;
  private int height;
```

These two values will be saved.

```
  public Square(int width, int height) {
    this.width = width;
    this.height = height;
  }
```

```
  public static void main(String[] args) {
    Square mySquare = new Square(50, 20);

    try {
      FileOutputStream fs = new FileOutputStream("foo.ser");
      ObjectOutputStream os = new ObjectOutputStream(fs);
      os.writeObject(mySquare);
      os.close();
    } catch (Exception ex) {
      ex.printStackTrace();
    }
  }
}
```

I/O operations can throw exceptions.

Connect to a file named "foo.ser" if it exists. If it doesn't, make a new file named "foo.ser."

Make an ObjectOutputStream chained to the connection stream.

Tell it to write the object.

Serialization is all or nothing.

Can you imagine what would happen if some of the object's state didn't save correctly?

Eeewww! That creeps me out just thinking about it! Like, what if a Dog comes back with no weight. Or no ears. Or the collar comes back size 3 instead of 30. That just can't be allowed!

```
import java.io.*;

public class Pond implements Serializable {

  private Duck duck = new Duck();

  public static void main(String[] args) {
    Pond myPond = new Pond();
    try {
      FileOutputStream fs = new FileOutputStream("Pond.ser");
      ObjectOutputStream os = new ObjectOutputStream(fs);

      os.writeObject(myPond);
      os.close();
    } catch (Exception ex) {
      ex.printStackTrace();
    }
  }
}
```

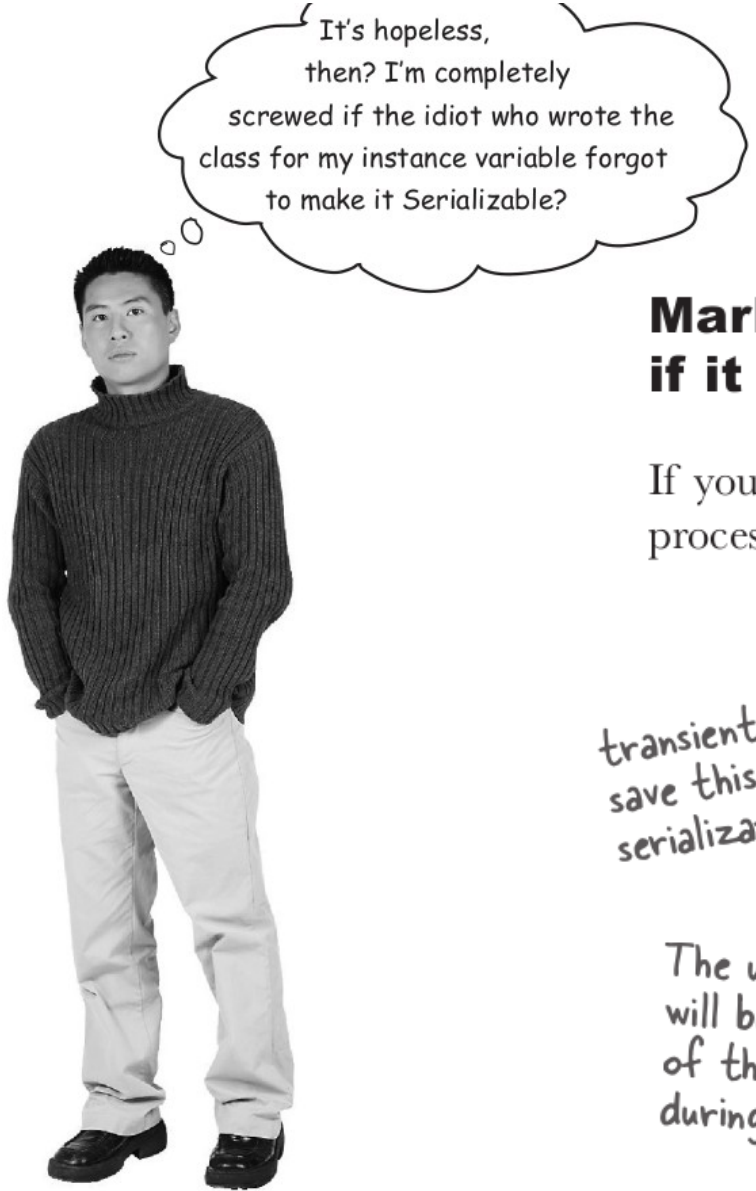
Pond objects can be serialized.

Class Pond has one instance variable, a Duck.

When you serialize myPond (a Pond object), its Duck instance variable automatically gets serialized.

When you try to run the main in class Pond:

```
File Edit Window Help Regret
% java Pond
java.io.NotSerializableException: Duck
        at Pond.main(Pond.java:13)
```



Mark an instance variable as transient if it can't (or shouldn't) be saved.

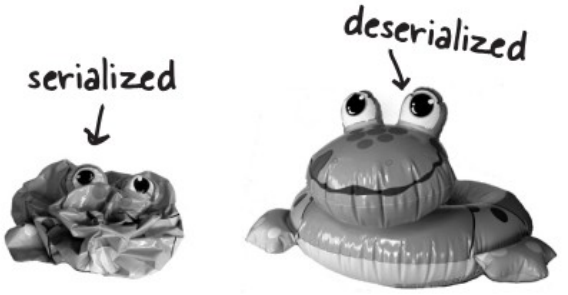
If you want an instance variable to be skipped by the serialization process, mark the variable with the **transient** keyword.

transient says, "don't save this variable during serialization; just skip it."

```
import java.net.*;
class Chat implements Serializable {
  transient String currentID;

  String userName;

  // more code
}
```

The userName variable will be saved as part of the object's state during serialization.

Deserialization: restoring an object



1 Make a FileInputStream

```
FileInputStream fileStream = new FileInputStream("MyGame.ser");
```

If the file "MyGame.ser" doesn't exist, you'll get an exception.

Make a FileInputStream object. The FileInputStream knows how to connect to an existing file.

2 Make an ObjectInputStream

```
ObjectInputStream os = new ObjectInputStream(fileStream);
```

ObjectInputStream lets you read objects, but it can't directly connect to a file. It needs to be chained to a connection stream, in this case a FileInputStream.

3 Read the objects

```
Object one = os.readObject();
Object two = os.readObject();
Object three = os.readObject();
```

Each time you say readObject(), you get the next object in the stream. So you'll read them back in the same order in which they were written. You'll get a big fat exception if you try to read more objects than you wrote.

4 Cast the objects

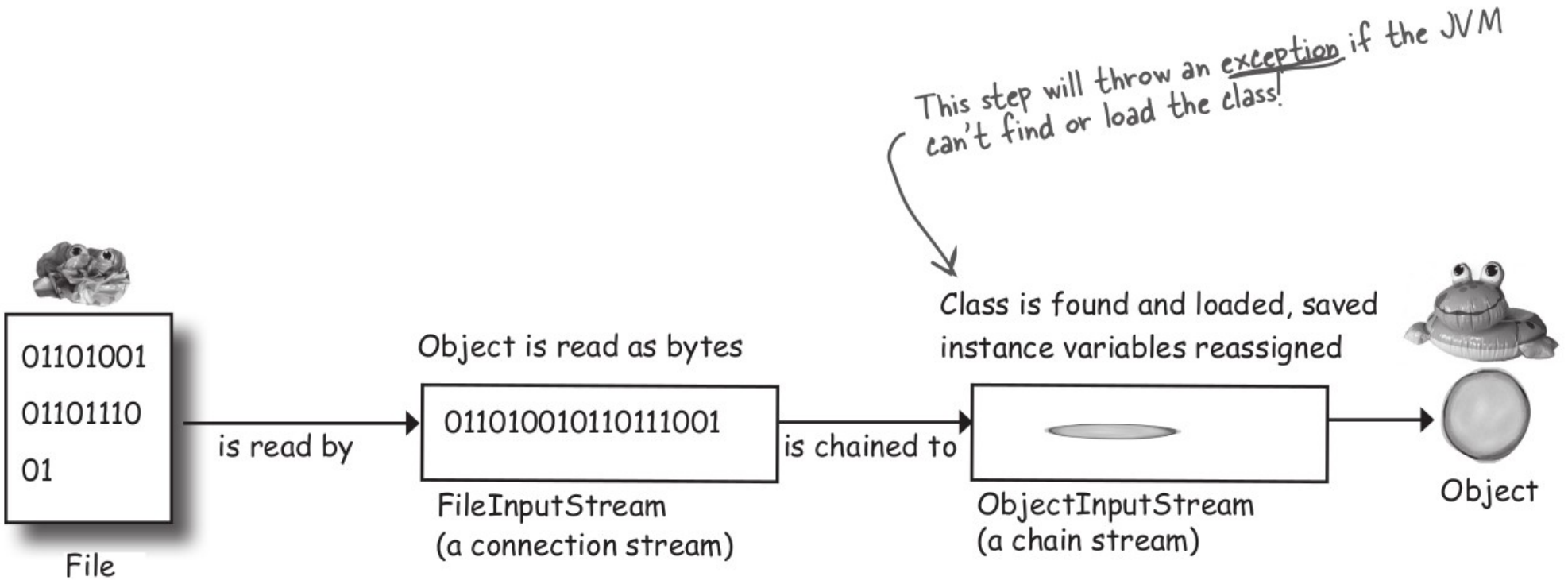
```
GameCharacter elf = (GameCharacter) one;
GameCharacter troll = (GameCharacter) two;
GameCharacter magician = (GameCharacter) three;
```

The return value of readObject() is type Object (just like with ArrayList), so you have to cast it back to the type you know it really is.

5 Close the ObjectInputStream

```
os.close();
```

Closing the stream at the top closes the ones underneath, so the FileInputStream (and the file) will close automatically.


This step will throw an exception if the JVM can't find or load the class!
01101001
01101110
01
File
is read by
Object is read as bytes
0110100101101110 01
FileInputStream
(a connection stream)
is chained to
Class is found and loaded, saved instance variables reassigned
ObjectInputStream
(a chain stream)
Object

1. The object is **read** from the stream.

2. The JVM determines (through info stored with the serialized object) the object's **class type**.

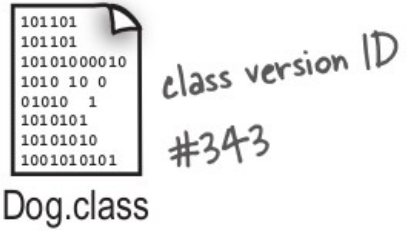
3. The JVM attempts to **find and load** the object's **class**. If the JVM can't find and/or load the class, the JVM throws an exception and the deserialization fails.

4. A new object is given space on the heap, but the **serialized object's constructor does NOT run**! Obviously, if the constructor ran, it would restore the state of the object to its original "new" state, and that's not what we want. We want the object to be restored to the state it had *when it was serialized*, not when it was first created.

5. If the object has a non-serializable class somewhere up its inheritance tree, the **constructor for that non-serializable class will run** along with any constructors above that (even if they're serializable). Once the constructor chaining begins, you can't stop it, which means all superclasses, beginning with the first non-serializable one, will reinitialize their state.

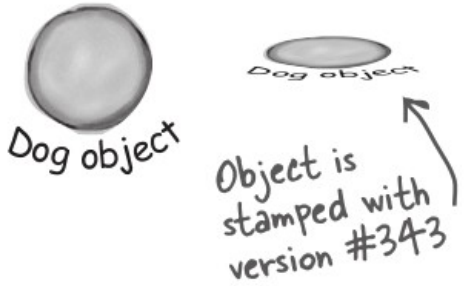
6. The object's **instance variables are given the values from the serialized state**. Transient variables are given a value of null for object references and defaults (0, false, etc.) for primitives.

1 You write a Dog class.



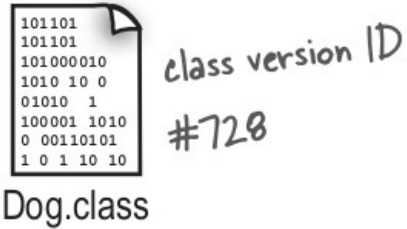
---

2 You serialize a Dog object using that class.

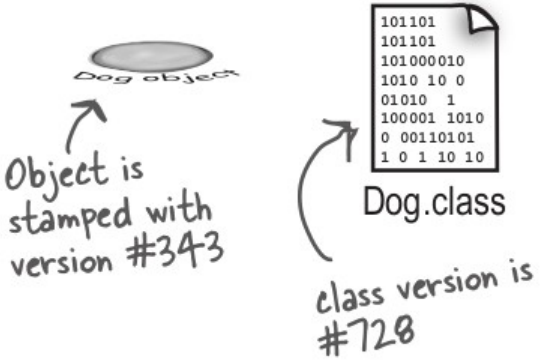


---

3 You change the Dog class.



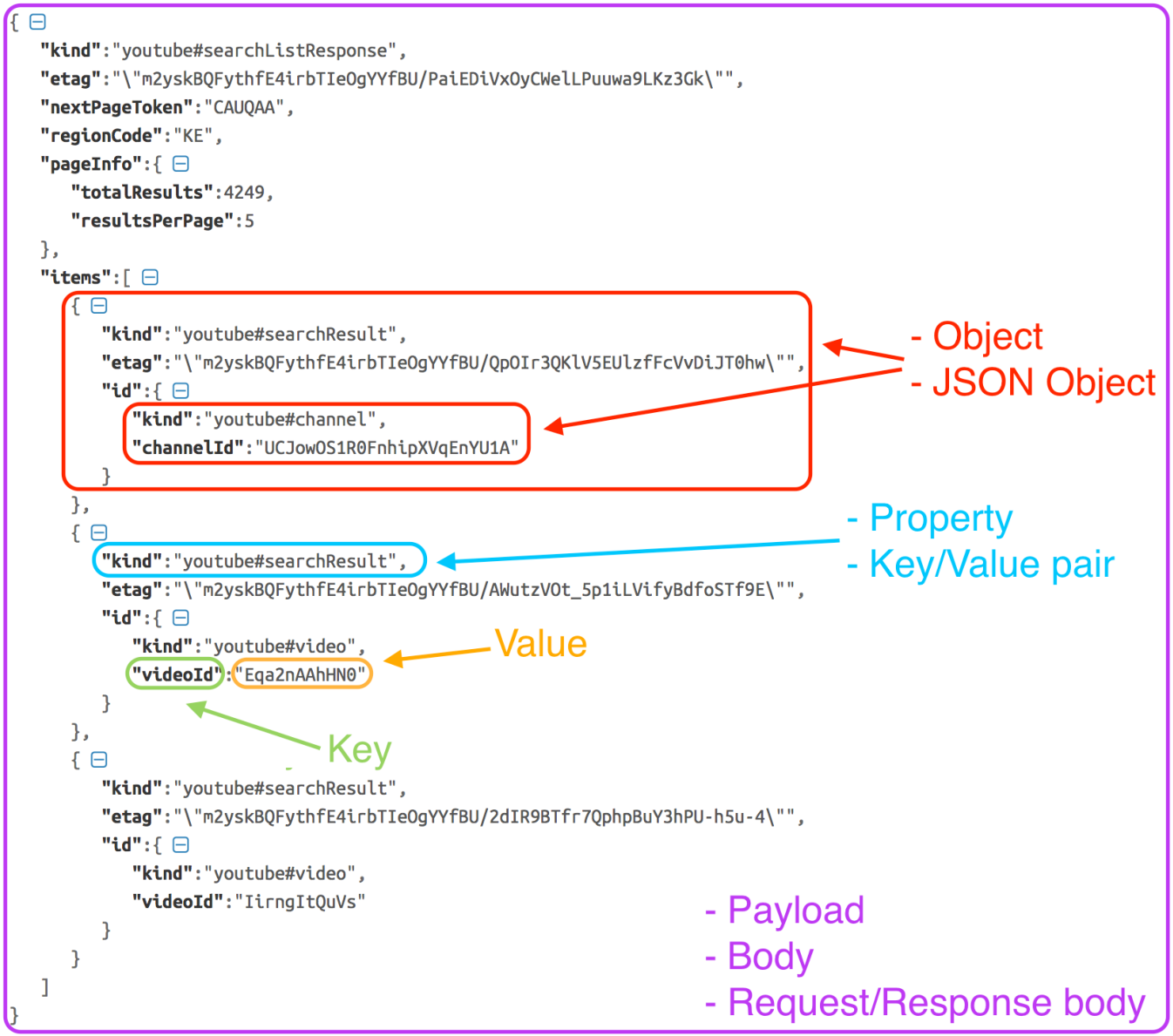
4 You deserialize a Dog object using the changed class.



---

5 Serialization fails!!

The JVM says, "you can't teach an old Dog new code."


{
"kind":"youtube#searchListResponse",
"etag":"\"m2yskBQFythfE4irbTIeOgYYfBU/PaiEDiVxOyCWelLPuuwa9LKz3Gk\"",
"nextPageToken":"CAUQAA",
"regionCode":"KE",
"pageInfo":{
"totalResults":4249,
"resultsPerPage":5
},
"items":[
{
"kind":"youtube#searchResult",
"etag":"\"m2yskBQFythfE4irbTIeOgYYfBU/QpOIr3QKlV5EUlzfFcVvDiJT0hw\"",
"id":{
"kind":"youtube#channel",
"channelId":"UCJowOS1R0FnhipXVqEnYU1A"
}
},
{
"kind":"youtube#searchResult",
"etag":"\"m2yskBQFythfE4irbTIeOgYYfBU/AWutzVOt_5p1iLVifyBdfoSTf9E\"",
"id":{
"kind":"youtube#video",
"videoId":"Eqa2nAAhHN0"
}
},
{
"kind":"youtube#searchResult",
"etag":"\"m2yskBQFythfE4irbTIeOgYYfBU/2dIR9BTfr7QphpBuY3hPU-h5u-4\"",
"id":{
"kind":"youtube#video",
"videoId":"IirngItQuVs"
}
}
]
}
- Object
- JSON Object
- Property
- Key/Value pair
Value
Key
- Payload
- Body
- Request/Response body

To write a String:

```
fileWriter.write("My first String to save");
```

```
import java.io.*;

class WriteAFile {
  public static void main(String[] args) {
    try {
      FileWriter writer = new FileWriter("Foo.txt");

      writer.write("hello foo!");

      writer.close();
    } catch (IOException ex) {
      ex.printStackTrace();
    }
  }
}
```

We need the java.io package for FileWriter.

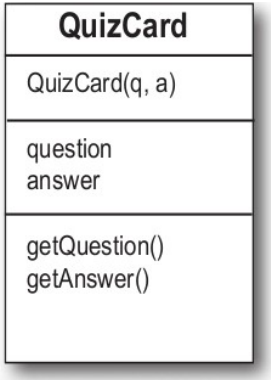
If the file "Foo.txt" does not exist, FileWriter will create it.

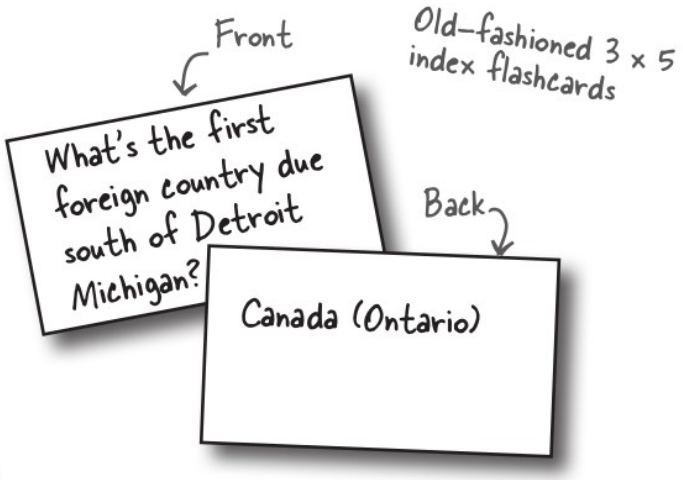
ALL the I/O stuff must be in a try/catch. Everything can throw an IOException!!

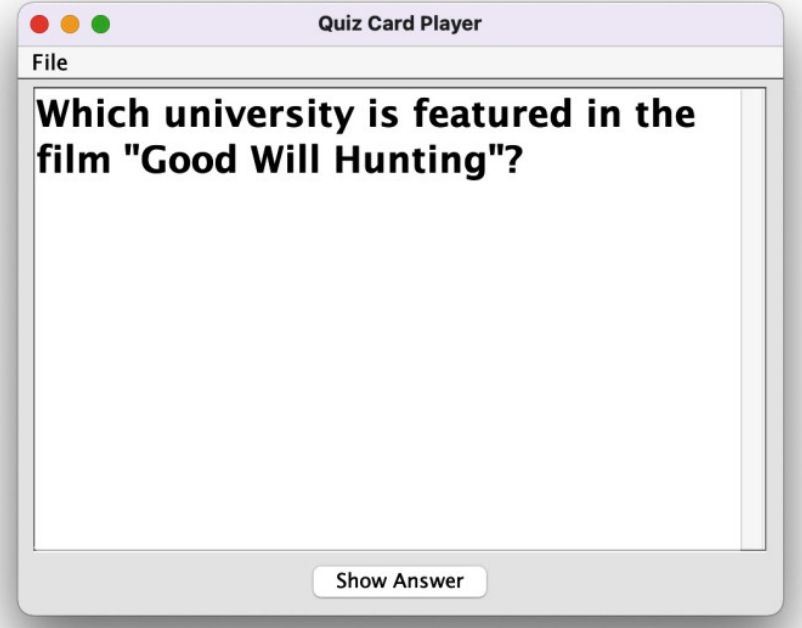
The write() method takes a String.

Close it when you're done!


Quiz Card Builder
File
Question:
Which university is featured in the film "Good Will Hunting"?
Answer:
M.I.T.
Next Card


QuizCard
QuizCard(q, a)
question
answer
getQuestion()
getAnswer()


Front
Old-fashioned 3 x 5 index flashcards
What's the first foreign country due south of Detroit Michigan?
Back
Canada (Ontario)


Quiz Card Player
File
Which university is featured in the film "Good Will Hunting"?
Show Answer

Some things you can do with a File object:

① **Make a File object representing an existing file**

```
File f = new File("MyCode.txt");
```

② **Make a new directory**

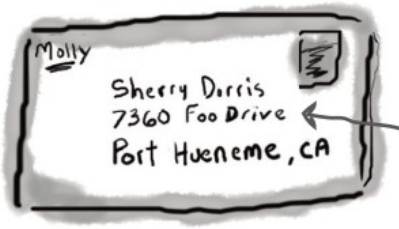
```
File dir = new File("Chapter7");
dir.mkdir();
```

③ **List the contents of a directory**

```
if (dir.isDirectory()) {
  String[] dirContents = dir.list();
  for (String dirContent : dirContents) {
    System.out.println(dirContent);
  }
}
```

④ **Delete a file or directory (returns true if successful)**

```
boolean isDeleted = f.delete();
```



An address is NOT the same as the actual house! A File object is like a street address... it represents the name and location of a particular file, but it isn't the file itself.

A File object represents the filename "GameFile.txt."

GameFile.txt

50,Elf,bow, sword,dust
200,Troll,bare hands,big ax
120,Magician,spells,invisibility

A File object does NOT represent (or give you direct access to) the data inside the file!

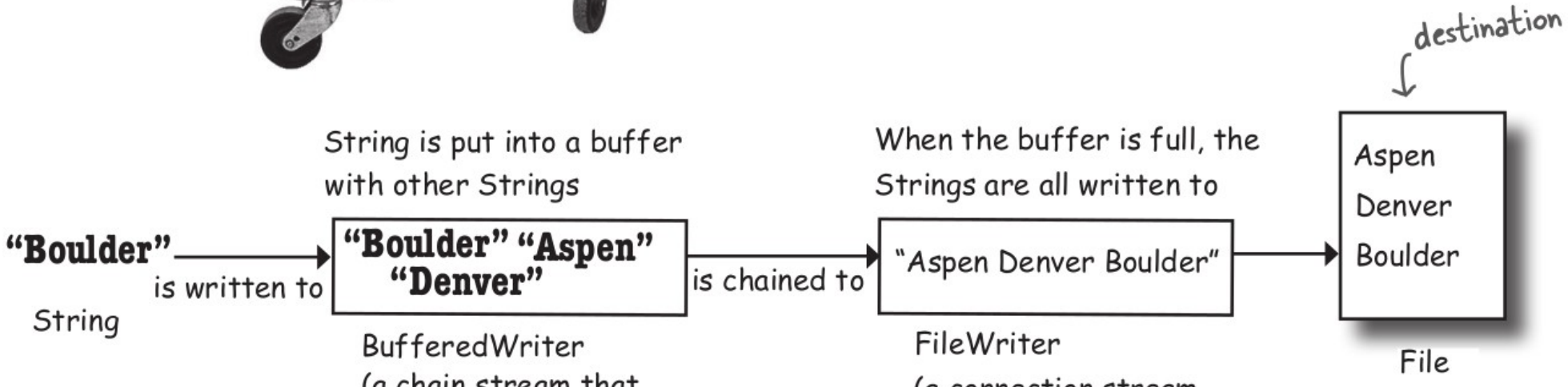
The method that does the actual file writing (called by the SaveMenuListener's event handler). The argument is the 'File' object the user is saving. We'll look at the File class on the next page.

We chain a BufferedWriter on to a new FileWriter to make writing more efficient. (We'll talk about that in a few pages.)

```
private void saveFile(File file) {
  try {
    BufferedWriter writer = new BufferedWriter(new FileWriter(file));
    for (QuizCard card : cardList) {
      writer.write(card.getQuestion() + "/");
      writer.write(card.getAnswer() + "\n");
    }
    writer.close();
  } catch (IOException e) {
    System.out.println("Couldn't write the cardList out: " + e.getMessage());
  }
}
```

Walk through the ArrayList of cards and write them out, one card per line, with the question and answer separated by a "/", and then add a newline character ("\n").

Buffers give you a temporary holding place to group things until the holder (like the cart) is full. You get to make far fewer trips when you use a buffer.



```
import java.io.*;

class ReadAFile {
  public static void main(String[] args) {
    try {
      File myFile = new File("MyText.txt");
      FileReader fileReader = new FileReader(myFile);

      BufferedReader reader = new BufferedReader(fileReader);

      String line;
      while ((line = reader.readLine()) != null) {
        System.out.println(line);
      }
      reader.close();

    } catch (IOException e) {
      e.printStackTrace();
    }
  }
}
```

Don't forget the import.

A FileReader is a connection stream for characters that connects to a text file.

Chain the FileReader to a BufferedReader for more efficient reading. It'll go back to the file to read only when the buffer is empty (because the program has read everything in it).

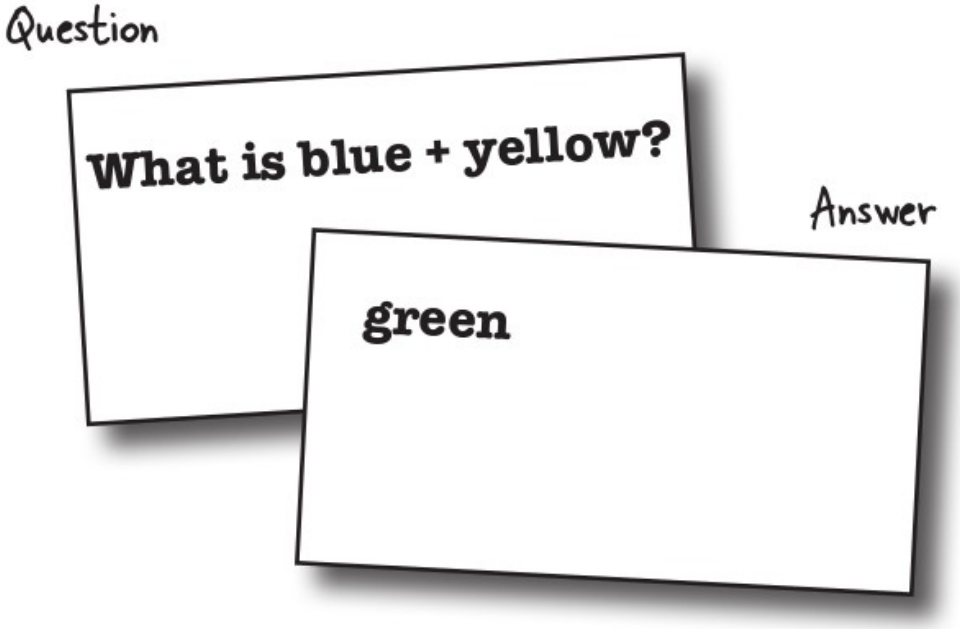
Make a String variable to hold each line as the line is read

This says, "Read a line of text, and assign it to the String variable "line." While that variable is not null (because there WAS something to read), print out the line that was just read."

Or another way of saying it, "While there are still lines to read, read them and print them."

Parsing with String split()

Imagine you have a flashcard like this:



Saved in a question file like this:

What is blue + yellow?/green
What is red + blue?/purple

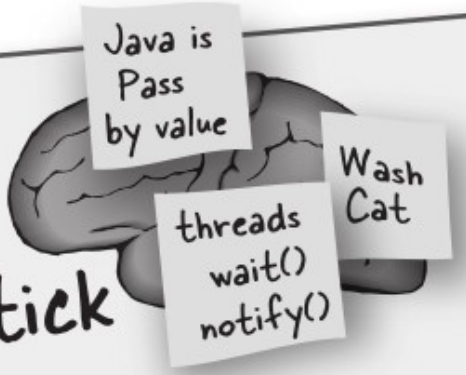
In the QuizCardPlayer app, this is what a single line looks like when it's read in from the file.

```
String toTest = "What is blue + yellow?/green";

String[] result = toTest.split("/");

for (String token : result) {

  System.out.println(token);

}
```

The split() method takes the "/" and uses it to break apart the String into (in this case) two pieces, token 1 and token 2. (Note: split() is FAR more powerful than what we're using it for here. It can do extremely complex parsing with filters, wildcards, etc.)

Loop through the array and print each token (piece). In this example, there are only two tokens: "What is blue + yellow?" and "green."

Make it Stick



Roses are first, violets are next.

***Readers** and **Writers** are only for **text**.*

① Import Path, Paths, and Files:

```
import java.nio.file.*;
```

② Make a Path object using the Paths class:

```
Path myPath = Paths.get("MyFile.txt");
```

A Path object is used to locate a file on a computer (i.e., in the file system). A path can be used to locate files in the current directory or in other directories.

Or, if the file is in a subdirectory like: /myApp/files/MyFile.txt :

```
Path myPath = Paths.get("/myApp", "files", "MyFile.txt");
```

The "/" in "/myApp" is called the name-separator. Depending on which OS you're using, your name-separator might be different; for example, it might be "\".

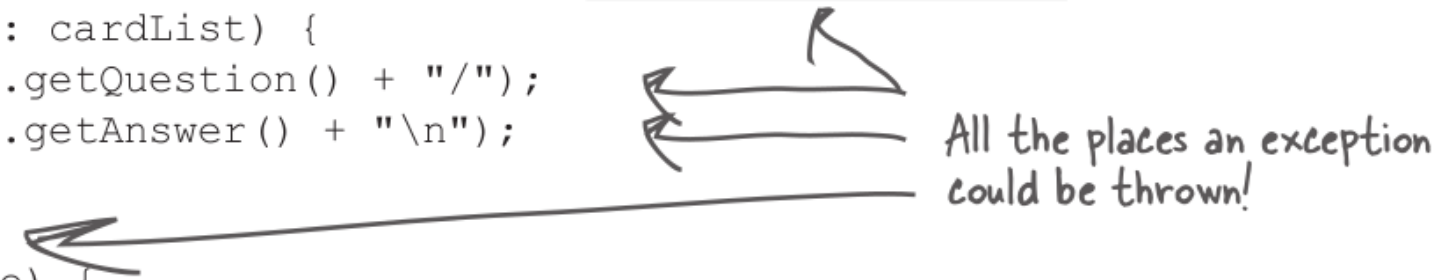
③ Make a new BufferedWriter using a Path and the Files class:

```
BufferedWriter writer = Files.newBufferedWriter(myPath);
```

Somewhere—under the covers—some method is saying:

```
BufferedWriter writer =
    new BufferedWriter(..)
```

```
private void saveFile(File file) {
  try {
    BufferedWriter writer = new BufferedWriter(new FileWriter(file));
    for (QuizCard card : cardList) {
      writer.write(card.getQuestion() + "/");
      writer.write(card.getAnswer() + "\n");
    }
    writer.close();
  } catch (IOException e) {
    System.out.println("Couldn't write the cardList out: " + e.getMessage());
  }
}
```



```
private void saveFile(File file) {
  BufferedWriter writer = null;
  try {
    writer = new BufferedWriter(new FileWriter(file));
    for (QuizCard card : cardList) {
      writer.write(card.getQuestion() + "/");
      writer.write(card.getAnswer() + "\n");
    }
    writer.close();
  } catch (IOException e) {
    System.out.println("Couldn't write the cardList out: " + e.getMessage());
  } finally {
    try {
      writer.close();
    } catch (Exception e) {
      System.out.println("Couldn't close writer: " + e.getMessage());
    }
  }
```

We had to declare the writer reference outside of the try block so that it's visible in the finally block.

Yup, we had to put the close() in yet another try-catch block!

```
private void saveFile(File file) {
  try (BufferedWriter writer =
         new BufferedWriter(new FileWriter(file))) {

    for (QuizCard card : cardList) {
      writer.write(card.getQuestion() + "/");
      writer.write(card.getAnswer() + "\n");
    }

  } catch (IOException e) {
    System.out.println("Couldn't write the cardList out: " + e.getMessage());
  }
}
```

Modern, try-with-resources code

Inside the parentheses, declare an object whose type implements Autocloseable:

```
try (BufferedWriter writer =
       new BufferedWriter(new FileWriter(file))) {
```

Like all of the I/O classes we've been using this chapter, BufferedWriter implements Autocloseable.

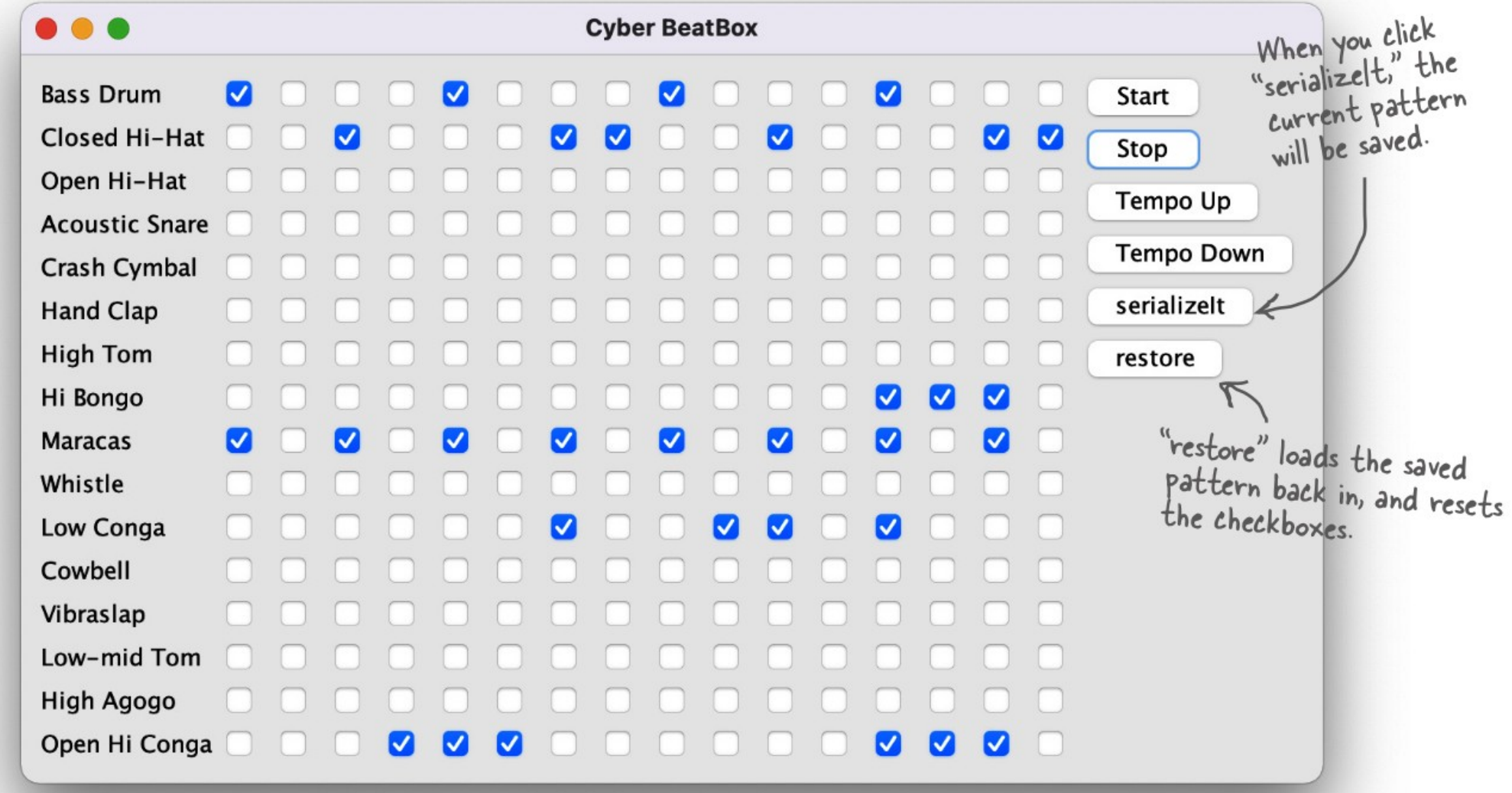
ONLY classes that implement Autocloseable can be used in TWR statements!

- You can declare and use more than one I/O resource in a single TWR block:

```
try (BufferedWriter writer =
       new BufferedWriter(new FileWriter(file));
     BufferedReader reader =
       new BufferedReader(new FileReader(file))) {
```

Separate the resources using semicolons, ";".

- If you declare more than one resource, they will be closed in the order OPPOSITE to which they were declared; i.e., first declared is last closed.
- If you add catch or finally blocks, the system will handle multiple close() invocations gracefully.


Cyber BeatBox
Bass Drum
Closed Hi-Hat
Open Hi-Hat
Acoustic Snare
Crash Cymbal
Hand Clap
High Tom
Hi Bongo
Maracas
Whistle
Low Conga
Cowbell
Vibraslap
Low-mid Tom
High Agogo
Open Hi Conga
Start
Stop
Tempo Up
Tempo Down
serializeIt
restore
When you click "serializeIt," the current pattern will be saved.
"restore" loads the saved pattern back in, and resets the checkboxes.

Serializing a pattern

This is a method in the BeatBox code. We can call this from a lambda expression when we add an ActionListener to the serializeIt button, or create an ActionListener inner class that calls this.

```
private void writeFile() {

  boolean[] checkboxState = new boolean[256];

  for (int i = 0; i < 256; i++) {
    JCheckBox check = checkboxList.get(i);
    if (check.isSelected()) {
      checkboxState[i] = true;
    }
  }

  try (ObjectOutputStream os =
         new ObjectOutputStream(new FileOutputStream("Checkbox.ser"))) {
    os.writeObject(checkboxState);
  } catch (IOException e) {
    e.printStackTrace();
  }
}
```

Make a boolean array to hold the state of each checkbox.

Walk through the checkboxList (ArrayList of checkboxes), get the state of each one, and add it to the boolean array.

Try-with-resources

This part's a piece of cake. Just write/serialize the one boolean array!

Restoring a pattern

This is another method in the BeatBox class.

```
private void readFile() {
  boolean[] checkboxState = null;
  try (ObjectInputStream is =
        new ObjectInputStream(new FileInputStream("Checkbox.ser"))) {
    checkboxState = (boolean[]) is.readObject();
  } catch (Exception e) {
    e.printStackTrace();
  }

  for (int i = 0; i < 256; i++) {
    JCheckBox check = checkboxList.get(i);
    check.setSelected(checkboxState[i]);
  }

  sequencer.stop();
  buildTrackAndStart();
}
```

Try-with-resources

Read the single object in the file (the boolean array) and cast it back to a boolean array (remember, readObject() returns a reference of type Object).

Now restore the state of each of the checkboxes in the ArrayList of actual JCheckBox objects (checkboxList).

Now stop whatever is currently playing, and rebuild the sequence using the new state of the checkboxes in the ArrayList.

What's Legal?

Circle the code fragments that would compile (assuming they're within a legal class).

→ Yours to solve.

```
FileReader fileReader = new FileReader();
BufferedReader reader = new BufferedReader(fileReader);
```

```
FileOutputStream f = new FileOutputStream("Foo.ser");
ObjectOutputStream os = new ObjectOutputStream(f);
```

```
BufferedReader reader = new BufferedReader(new FileReader(file));
String line;
while ((line = reader.readLine()) != null) {
  makeCard(line);
}
```

```
FileOutputStream f = new FileOutputStream("Game.ser");
ObjectInputStream is = new ObjectInputStream(f);
GameCharacter oneAgain = (GameCharacter) is.readObject();
```

What's Legal?

Circle the code fragments that would compile (assuming they're within a legal class).

→ Yours to solve.

```
FileReader fileReader = new FileReader();
BufferedReader reader = new BufferedReader(fileReader);
```

```
FileOutputStream f = new FileOutputStream("Foo.ser");
ObjectOutputStream os = new ObjectOutputStream(f);
```

```
BufferedReader reader = new BufferedReader(new FileReader(file));
String line;
while ((line = reader.readLine()) != null) {
  makeCard(line);
}
```

```
FileOutputStream f = new FileOutputStream("Game.ser");
ObjectInputStream is = new ObjectInputStream(f);
GameCharacter oneAgain = (GameCharacter) is.readObject();
```

1. Serialization is appropriate when saving data for non-Java programs to use.

2. Object state can be saved only by using serialization.

3. ObjectOutputStream is a class used to save serializable objects.

4. Chain streams can be used on their own or with connection streams.

5. A single call to writeObject() can cause many objects to be saved.

6. All classes are serializable by default.

7. The java.nio.file.Path class can be used to locate files.

8. If a superclass is not serializable, then the subclass can't be serializable.

9. Only classes that implement AutoCloseable can be used in try-with-resources statements.

True or False

1. Serialization is appropriate when saving data for non-Java programs to use. false
2. Object state can be saved only by using serialization. false
3. ObjectOutputStream is a class used to save serializable objects. true
4. Chain streams can be used on their own or with connection streams. false
5. A single call to writeObject() can cause many objects to be saved. true
6. All classes are serializable by default. false
7. The java.nio.file.Path class can be used to locate files. false
8. If a superclass is not serializable, then the subclass can't be serializable. false
9. Only classes that implement AutoCloseable can be used in try-with-resources statements. true

10. When an object is deserialized, its constructor does not run.

11. Both serialization and saving to a text file can throw exceptions.

12. BufferedWriters can be chained to FileWriters.

13. File objects represent files, but not directories.

14. You can't force a buffer to send its data before it's full.

15. Both file readers and file writers can optionally be buffered.

16. The methods on the Files class let you operate on files and directories.

17. Try-with-resources statements cannot include explicit finally blocks.

10. When an object is deserialized, its constructor does not run. true

11. Both serialization and saving to a text file can throw exceptions. true

12. BufferedWriters can be chained to FileWriters. true

13. File objects represent files, but not directories. false

14. You can't force a buffer to send its data before it's full. false

15. Both file readers and file writers can optionally be buffered. true

16. The methods on the Files class let you operate on files and directories. true

17. Try-with-resources statements cannot include explicit finally blocks. false